

A horizontal bar with a gold-to-white gradient is centered on the page. A large black left bracket is on the left, and a large gold right bracket is on the right. A thin gold circle is partially visible behind the left side of the bar.

SWE 213

Boolean Functions (Solve ?)

■ Examples:

$$F1 = xyz'$$

$$F2 = x + y'z$$

$$F4 = xy' + x'z$$

$$F3 = (x'y'z) + (x'yz) + (xy')$$

SOP,
minterm

x	y	z	F1	F2	F3	F4
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	0	0	0	0
0	1	1	0	0	1	1
1	0	0	0	1	1	1
1	0	1	0	1	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	0

From the truth table, $F3 = F4$.

Can you also prove by algebraic manipulation that $F3 = F4$?

Try it yourself: simplify the following equations

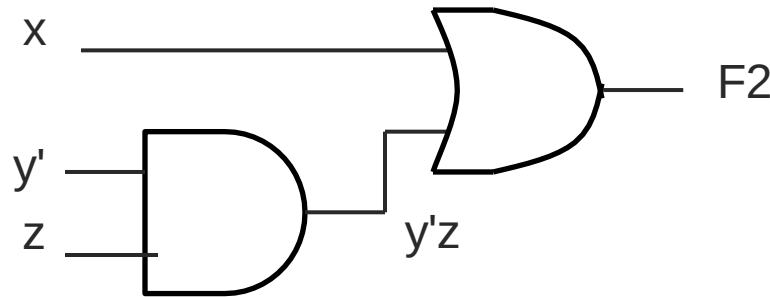
$$1. f(a,b,c,d) = abc + abd + a'bc' + cd + bd'$$

TABLE 2-1
Postulates and Theorems of Boolean Algebra

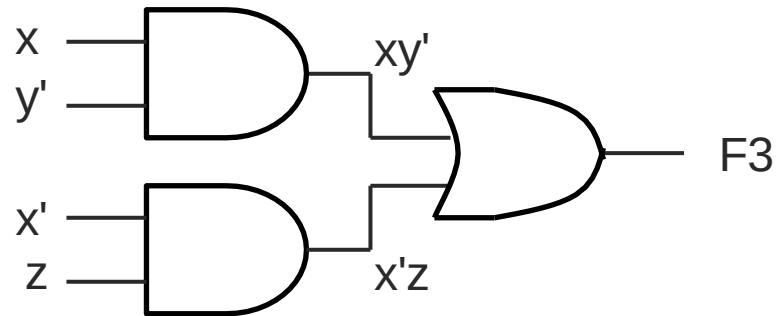
Postulate 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
Postulate 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x + x = x$	(b) $x \cdot x = x$
Theorem 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$
Theorem 3, involution	$(x')' = x$	
Postulate 3, commutative	(a) $x + y = y + x$	(b) $xy = yx$
Theorem 4, associative	(a) $x + (y + z) = (x + y) + z$	(b) $x(yz) = (xy)z$
Postulate 4, distributive	(a) $x(y + z) = xy + xz$	(b) $x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a) $(x + y)' = x'y'$	(b) $(xy)' = x' + y'$
Theorem 6, absorption	(a) $x + xy = x$	(b) $x(x + y) = x$

Drawing Logic Circuit

(ii) $F2 = x + y'z$ (can assume that variables and their complements are available)

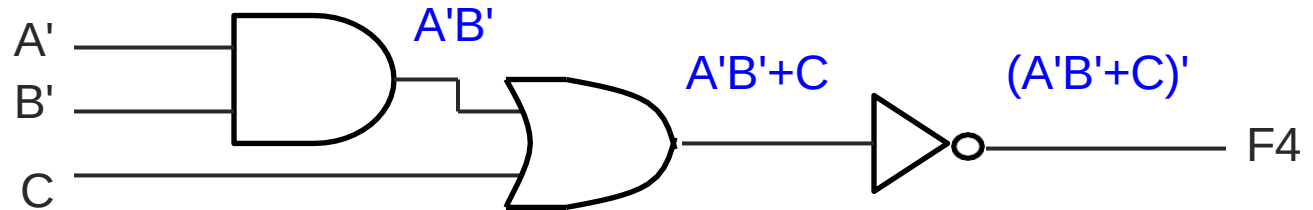


(iii) $F3 = xy' + x'z$



Analysing Logic Circuit

- When a logic circuit is provided, we can analyse the circuit to obtain the logic expression.
- Example: What is the Boolean expression of F4?



$$F4 = (A'B'+C)'$$



[Review

- If expression $F2 = x + y'z$ is given, below is its truth table (obtained by “AND”ing y' with z and then “OR”ing the product with x .)

- So from truth table, considering the 1's we can say,
 $F2 = x'y'z + xy'z' + xy'z + xyz' + xyz$
which when simplified using postulates, will result to,
 $F2 = x + y'z$

X	Y	Z	F2
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$\text{so } \underbrace{x + y'z}_{\text{Simplified expression}} = \underbrace{x'y'z + xy'z' + xy'z + xyz' + xyz}_{\text{SOP Canonical Format of the expression}}$$

Simplified
expression

SOP Canonical Format of the
expression

Review: Solve it yourself

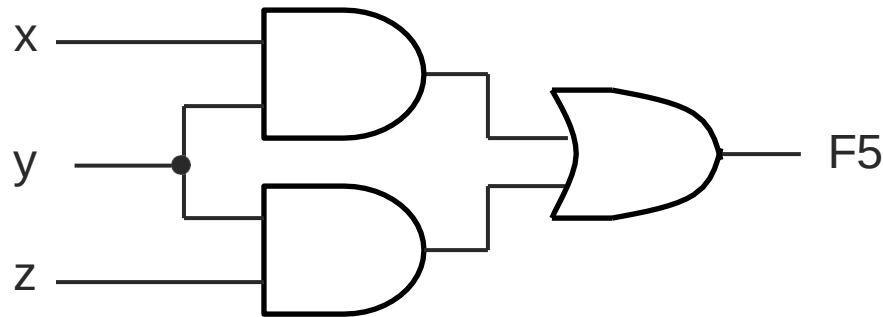
Q1. Draw a logic circuit for $BD + BE + D'F$

Q2. Draw a logic circuit for

$$A'BC + B'CD + BC'D + ABD'$$

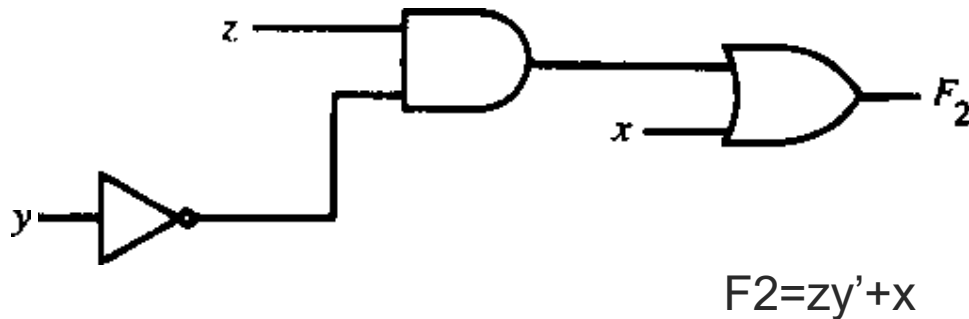
Review: Solve it yourself

- What is Boolean expression of F5?



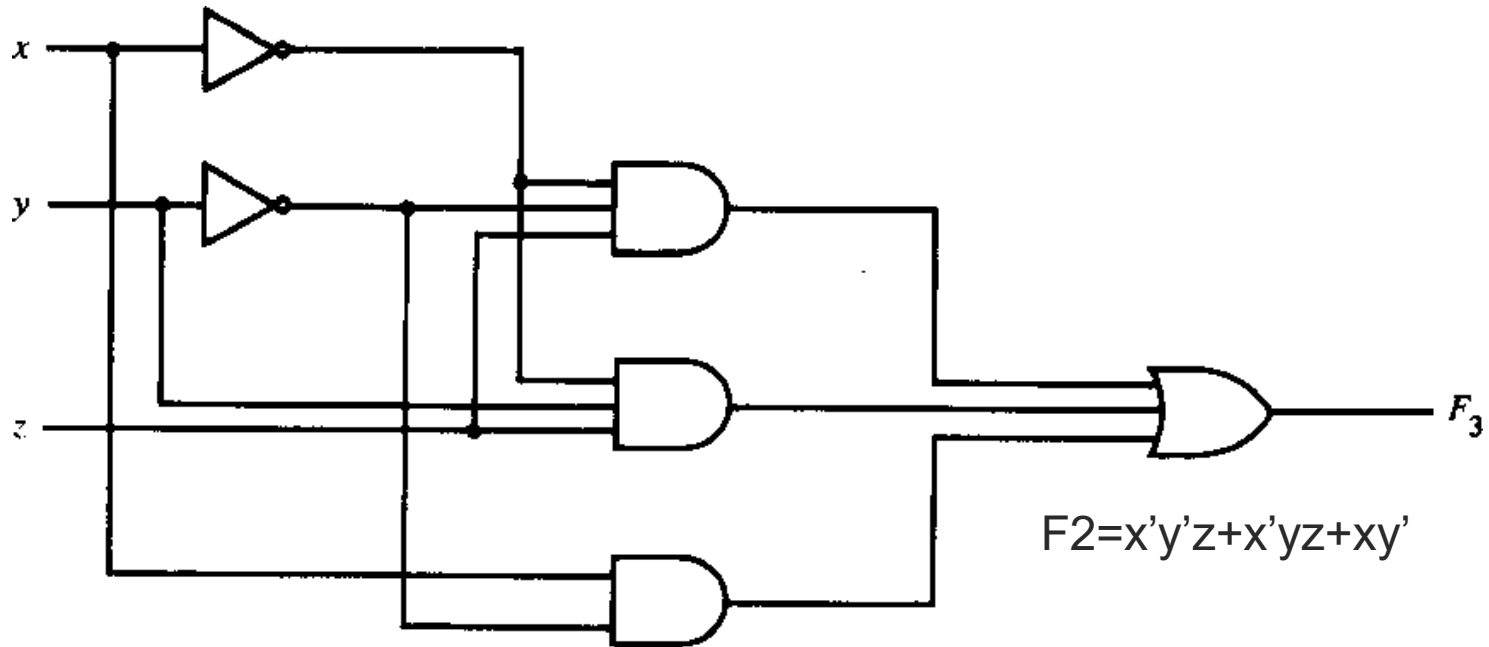
[Review: Solve it yourself]

- What is Boolean expression of F_2 ?



[Review: Solve it yourself]

- What is Boolean expression of F_3 ?



Standard Forms

- Two Standard Forms:
Sum-of-Products and *Product-of-Sums*
- **Literals**: a variable on its own or in its complemented form. Examples: x , x' , y , y'
- **Product Term**: a single literal or a logical product (AND) of several literals.
Examples: x , xyz' , $A'B$, AB

Standard Forms

- **Sum Term**: a single literal or a logical sum (OR) of several literals.
Examples: x , $x+y+z'$, $A'+B$, $A+B$
- **Sum-of-Products (SOP) Expression**: a product term or a logical sum (OR) of several product terms.
Examples: x , $x+yz'$, $xy'+x'yz$, $AB+A'B'$
- **Product-of-Sums (POS) Expression**: a sum term or a logical product (AND) of several sum terms.
Examples: x , $x(y+z')$, $(x+y')(x'+y+z)$, $(A+B)(A'+B')$
- Every boolean expression can either be expressed as sum-of-products or product-of-sums expression.

Examples:

SOP: $x'y + xy' + xyz$

POS: $(x + y')(x' + y)(x' +$

$z')$

Note: SOP and POS are complements

Minterm

- Consider two binary variables x, y .
- Each variable may appear as itself or in complemented form as literals (i.e. x, x' & y, y')
- For two variables, there are four possible combinations with the AND operator, namely:
$$x'y', x'y, xy', xy$$
- These product terms are called the *minterms*.
- A **minterm** of n variables is the product of n literals from the different variables.
- In general, n variables can give 2^n minterms.

Maxterm

- In a similar fashion, a **maxterm** of n variables is the sum of n literals from the different variables.

Examples: $x'+y'$, $x'+y$, $x+y'$, $x+y$

- In general, n variables can give 2^n maxterms.

Minterm & Maxterm

- The minterms and maxterms of 2 variables are denoted by m0 to m3 and M0 to M3 respectively:

Note:

$$m_0 = x'y'z'$$

$$m_0' = xyz$$

$$M_0 = (x+y+z)$$

$$m_0' = M_0$$

Minterms				Maxterms	
x	y	term	notation	term	notation
0	0	$x'y'$	m0	$x+y$	M0
0	1	$x'y$	m1	$x+y'$	M1
1	0	xy'	m2	$x'+y$	M2
1	1	xy	m3	$x'+y'$	M3

Minterm number is the decimal representation

Each minterm is the complement of the corresponding maxterm:

Example: $m_2 = xy'$

$$m_2' = (xy')' = x' + (y')' = x' + y = M_2$$



Minterms and Maxterms

Minterms					Maxterms	
<i>x</i>	<i>y</i>	<i>z</i>	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Canonical Form: Sum of Minterms

Obtain Sum-of-Minterms by gathering/summing the minterms of the function (where result is a 1)

$$F1 = xyz' = \Sigma(m6)$$

$$F2 = x'y'z + xy'z' + xy'z + xyz' + xyz = \Sigma(m1, m4, m5, m6, m7)$$

$$F3 = x'y'z + x'yz + xy'z' + xy'z = \Sigma(m1, m3, m4, m5)$$

x	y	z	F1	F2	F3
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	1	0
1	1	1	0	1	0

[Canonical Form: Product of Maxterms]

- Maxterms are sum terms.
- **For Boolean functions, the maxterms of a function are the terms for which the result is 0.**
- Boolean functions can be expressed as Products-of-Maxterms.

Canonical Form: Product of Maxterms

E.g.: $F2 = \prod(M0, M2, M3) = (x+y+z)(x+y'+z)(x+y'+z')$

$$F3 = \prod(M0, M2, M6, M7)$$

$$= (x+y+z)(x+y'+z)(x'+y'+z)(x'+y'+z')$$

x	y	z	F1	F2	F3
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	1	0
1	1	1	0	1	0

SOP
->POS

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



- Considering the 1's -> $f_1 = x'y'z + xy'z' + xyz' = m_1 + m_4 + m_7$

$$= \sum (m_1, m_4, m_7)$$
- Considering the 0's ->
 $f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz' = m_0 + m_2 + m_3 + m_5 + m_6$
- $(f_1')' = f_1 = (x+y+z)(x+y'+z)(x+y'+z')(x'+y+z')(x'+y'+z)$
- $f_1 = M_0 M_1 M_2 M_5 M_6 = \prod (M_0, M_2, M_3, M_5, M_6)$

Try writing f_2 in (i) SOP and (ii) POS

[Solution]

- SOP: $F2 = x'yz + xy'z + xyz' + xyz$
- POS: $F2 = (x+y+z)(x+y+z')(x+y'+z)(x'+y+z)$

Canonical Form: Product of Maxterms

- Why is this so? Take F2 as an example.

$$F2 = \Sigma(m1, m4, m5, m6, m7)$$

- The complement function of F2 is:

$$\begin{aligned} F2' &= \Sigma(m0, m2, m3) \\ &= m0 + m2 + m3 \end{aligned}$$

Therefore:

$$\begin{aligned} F2 &= (m0 + m2 + m3)' \\ &= \mathbf{m0' \cdot m2' \cdot m3'} \\ &= \mathbf{M0 \cdot M2 \cdot M3} \\ &= \Pi(M0, M2, M3) \end{aligned}$$

DeMorgan
 $m x' = M x$

- Every Boolean function can be expressed as either Sum-of-Minterms or Product-of-Maxterms.

x	y	z	F2	F2'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

Comparison between SOP and POS

- **Complement of a function**
= original function-minterms
= missing miniterms
- $m_j' = M_j$
- If $f(x,y,z) = \sum(1,3,6,7) = \prod(0,2,4,5)$

Function can be expressed in both SOP and POS.
Step 1: interchange $\sum$ with $\prod$
Step 2: list missing numbers from the original

Expressing Boolean Function in terms of Canonical SOP

Check if each term contains all variable, if not
AND $(x+x')$ if x is the missing term

$$F=A+B'C$$

$$=A(B+B')(C+C')+B'C(A+A')$$

$$=(AB+AB')(C+C')+B'C(A+A')$$

$$=AB(C+C')+AB'(C+C')+B'C(A+A')$$

$$=ABC+ABC'+\mathbf{AB'C}+AB'C'+\mathbf{AB'C}+A'B'C$$

$$=ABC+ABC'+AB'C+AB'C'+A'B'C$$

$$=\Sigma(1,4,5,6,7)$$

Try using Truth Table

Expressing Boolean function in terms of Canonical POS

1. Often distributive law $(x+yz)=(x+y)(x+z)$ is used
2. If then terms, like x , are missing, OR xx'

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) = (x + x')(y + x')(x + z)(y + z) \\ &= (1)(y + x')(x + z)(y + z) = (y + x')(x + z)(y + z) \end{aligned}$$

Each POS is missing a term so OR missing terms

$$= (y + x' + zz')(x + z + yy')(y + z + xx')$$

Again applying distributive law

$$= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') = \prod(0, 2, 4, 5)$$

Try using Truth Table

Expressing Boolean function in terms of Canonical SOP

- $F(w,x,y,z) = wy + x'z$

$$\begin{aligned}
 F(w,x,y,z) &= wy + x'z \\
 &= wy[(x+x')(z+z')] + x'z[(w+w')(y+y')] \\
 &= wy[xz + xz' + x'z + x'z'] + x'z[wy + wy' + w'y + w'y'] \\
 &= \overset{1}{w}\overset{1}{x}\overset{1}{y}\overset{1}{z} + \overset{1}{w}\overset{1}{x}\overset{1}{y}\overset{0}{z}' + \overset{1}{w}\overset{0}{x}'\overset{1}{y}\overset{1}{z} + \overset{1}{w}\overset{0}{x}'\overset{1}{y}\overset{0}{z}' + \overset{1}{w}\overset{0}{x}'\overset{0}{y}\overset{1}{z} + \overset{1}{w}\overset{0}{x}'\overset{0}{y}\overset{0}{z}' + \overset{0}{w}'\overset{1}{x}\overset{1}{y}\overset{1}{z} + \overset{0}{w}'\overset{1}{x}\overset{1}{y}\overset{0}{z}' + \overset{0}{w}'\overset{1}{x}\overset{0}{y}\overset{1}{z} + \overset{0}{w}'\overset{1}{x}\overset{0}{y}\overset{0}{z}' \\
 &= wxyz + wxyz' + wx'y z + wx'y z' + wx'y' z + wx'y' z' + w'x y z + w'x y z' + w'x' y z + w'x' y z' \\
 &= \sum(15, 14, 11, 10, 9, 3, 1)
 \end{aligned}$$

Expressing Boolean function in terms of Canonical POS

- $F(w,x,y,z) = wy + x'z$

$$F(w,x,y,z) = wy + x'z$$

$$= (wy + x') (wy + z)$$

$$= (w+x') (y+x') (w+z) (y+z)$$

$$= (w+x'+yy') (y+x'+ww') (w+z+xx') (y+z+ww')$$

$$= (w+x'+y) (w+x'+y') (y+x'+w) (y+x'+w')$$

$$(w+z+x) (w+z+x') (y+z+w) (y+z+w')$$

$$= (w+x'+y+zz') (w+x'+y'+zz') (y+x'+w+zz') (y+x'+w'+zz')$$

$$(w+z+x+yy') (w+z+x'+yy') (y+z+w+xx') (y+z+w'+xx')$$

$$= (w+x'+y+z) (w+x'+y+z') (w+x'+y'+z) (w+x'+y'+z')$$

$$(w+x'+y+z) (w+x'+y+z') (w'+x'+y+z) (w'+x'+y+z')$$

$$(w+x+y+z) (w+x+y'+z) (w+x'+y+z) (w+x'+y'+z)$$

$$(w+x+y+z) (w+x'+y+z) (w'+x+y+z) (w'+x'+y+z)$$

$$= \begin{pmatrix} 0 & 1 & 1 & 0 \\ w+x'+y'+z' \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 & 1 \\ w+x'+y+z' \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 1 \\ w'+x'+y+z' \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 0 \\ w+x'+y'+z \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 & 1 & 0 \\ w+x+y'+z \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 & 0 \\ w+x+y+z \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 & 0 \\ w+x'+y+z \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ w'+x+y+z \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 0 \\ w'+x'+y+z \end{pmatrix}$$

$$(7, 5, 13, 6, 0, 2, 4, 8, 12)$$

Solving it yourself

1. A Boolean function of 5 variables can have up to _____ minterms.
2. Given a Boolean function $F(A,B,C) = \sum m(0, 5, 7)$. Which of the following is correct?
 - a) $F'(A,B,C) = \sum m(0,5,7)$
 - b) $F(A,B,C) = \prod M(1,2,3,4,6)$
 - c) $F(A,B,C) = \prod M(0,5,7)$
 - d) $F'(A,B,C) = \sum m(1,2,3)$
 - e) None above
3. Given a Boolean function $F(x,y,z) = y' \cdot (x + z') + x' \cdot z$. Which of the following is correct?
 - a) $F(x,y,z) = \sum m(0,1)$
 - b) $F(x,y,z) = \sum m(0,1,4,5)$
 - c) $F(x,y,z) = \sum m(0,1,2,3,4)$
 - d) $F(x,y,z) = \sum m(0,1,3,4,5)$
 - e) $F(x,y,z) = \sum m(1,2,3,4,5)$

[Solution]

- 1. $2^5=32$
- 2. (b)
- 3.(d)

Working:

$$\begin{aligned} F(x,y,z) &= y' \cdot (x + z') + x' \cdot z = y'x + x'z \\ &= y'x(z+z') + x'z(y+y') \\ &= x'y'z' + x'y'z + x'yz + xy'z' + xy'z \\ &= m_0 + m_1 + m_3 + m_4 + m_5 \\ &= F(x,y,z) = \Sigma m(0,1,3,4,5) \end{aligned}$$

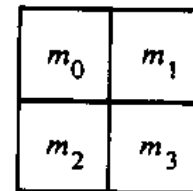
CSE215 Digital Logic Design

Karnaugh

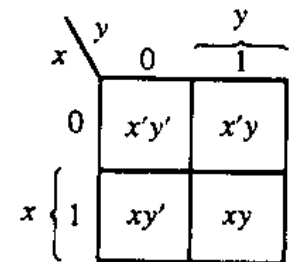
Maps

[The map method]

- The map method provides a simple procedure for minimizing Boolean functions.
- The map is made up of squares where each square represent a minterm.
- We represent each minterm of an equation by '1's
- We groups the adjacent '1's in groups of 2, groups of 4 or groups of 8 and so on.
- In those grouping try to find the common literal.
- The map method is often known as Karnaugh Map or Veitech Diagram. In short we will call it K-map



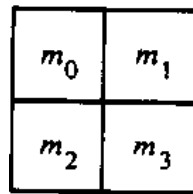
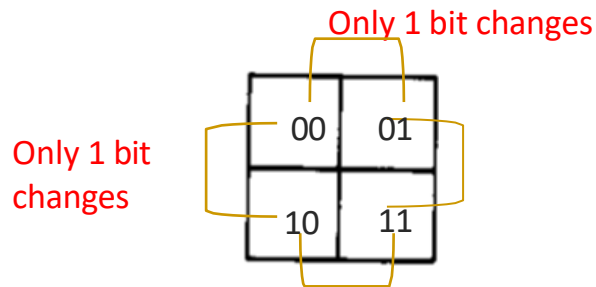
(a)



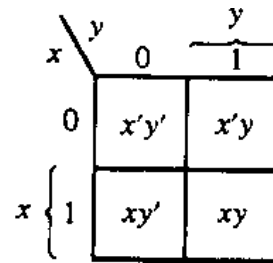
(b)

[Two variable map]

- There are 4 minterm of 2 variable; hence the map is four squares, one for each minterm.



(a)



(b)

Representing function in K-map

■ (a) $F1=xy$;

$x \backslash y$		y	
		0	1
x	0		
	1		1

(a) xy

■ (b) $F2=xy'+xy+x'y$

$$\begin{aligned} &=x(y'+y)+x'y \\ &=x+x'y \\ &=(x+x')(x+y) \\ &=(1)(x+y) \\ &=x+y \end{aligned}$$

$x \backslash y$		y	
		0	1
x	0		1
	1	1	1

(b) $x+y$

[Try it yourself]

$$F = xy' + x'y' + x'y,$$

Simplify the equation using k-map

[Solution]

■ $F = xy' + x'y' + x'y = x' + y'$

		y	
		0	1
x	0	1	1
	1	1	

[Three variable map]

- Sequence in 3-variable map is not in binary sequence but in reflected code. In reflected code, at a time only 1 bit changes from 0 to 1 and 1 to 0.
- Numbering of minterms follow binary numbers. Example m_5 is at 101 i.e. row 1 and column 01.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		yz			
		y			
		00	01	11	10
x	0	000	001	011	010
x	1	100	101	111	110
		z			

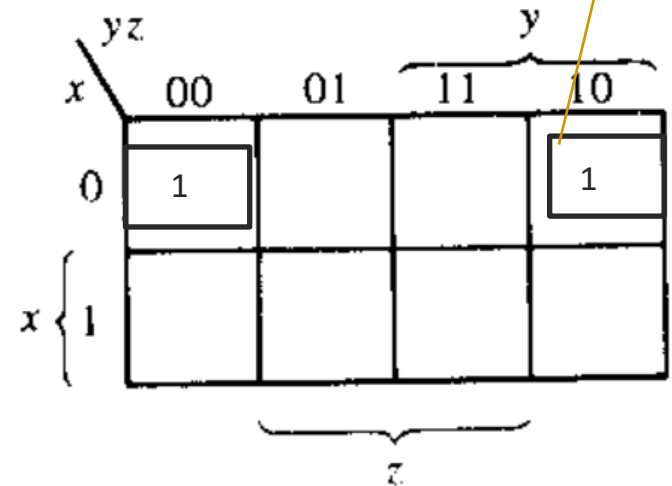
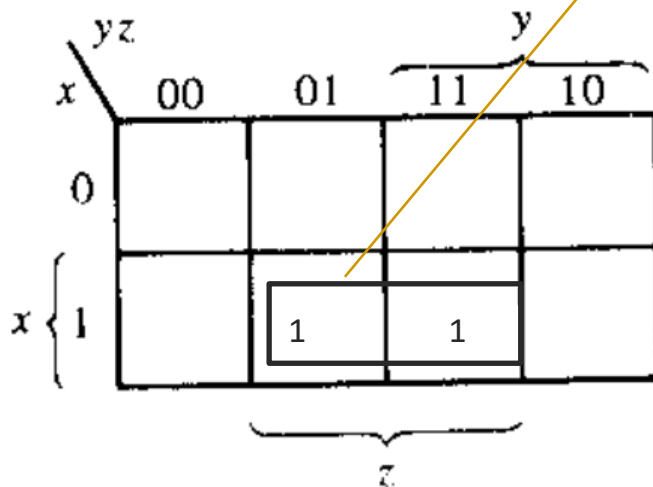
Note: Any 2 adjacent square differ by 1 variable (which is primed in one and unprimed in the other)

		yz			
		y			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
x	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

and unprimed in the other) and they are grouped, then any 2 minterms in the adjacent square are ORed, will remove the different term. As a result , we get a single ANDed term of only 2 common literals

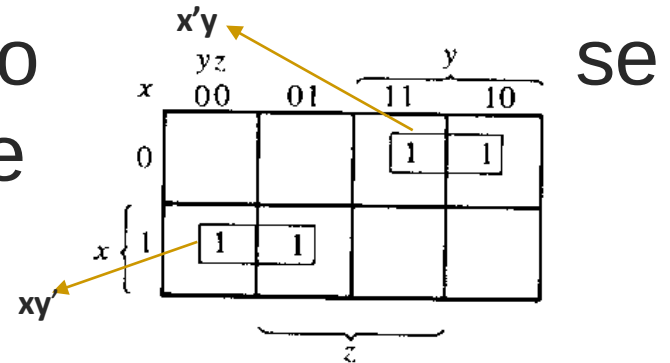
■ $F1 = \sum(5,7) = xy'z + xyz = xz(y' + y) = xz$

■ $F2 = \sum(0,2) = x'y'z' + x'yz' = x'z'(y' + y) = x'z'$



Simplifying functions using k-map

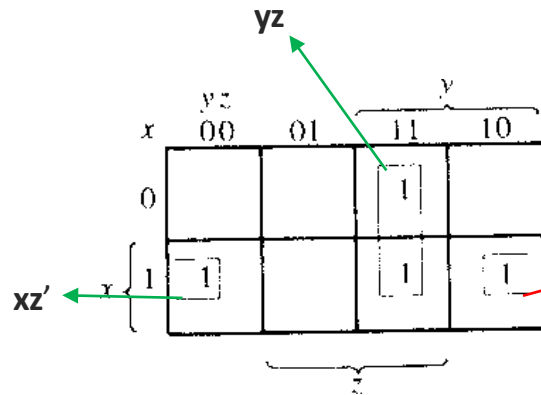
- $F = \sum(2,3,4,5) = x'yz + x'yz' + xy'z' + xy'z$
(note: in other words, $F = (\sum(010,011,100,101))$ so positions and group the $F = x'y + xy'$



- Grouping of 1 can be done in groups of two '1', four '1', eight '1'.... 2^n '1'.

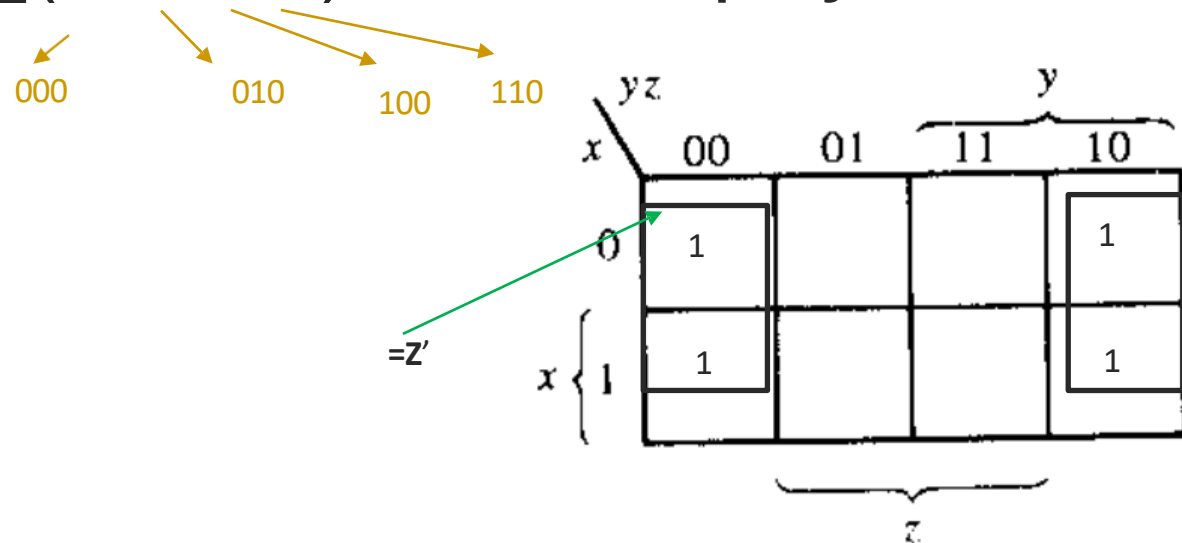
■ $F1 = x'yz + xy'z' + xyz + xyz' = xz' + yz$

011 100 111 110



Notice even 2 opposite edges of the map are adjacent, thus they can be grouped

- If $F = \sum(0, 2, 4, 6)$, then simplify it

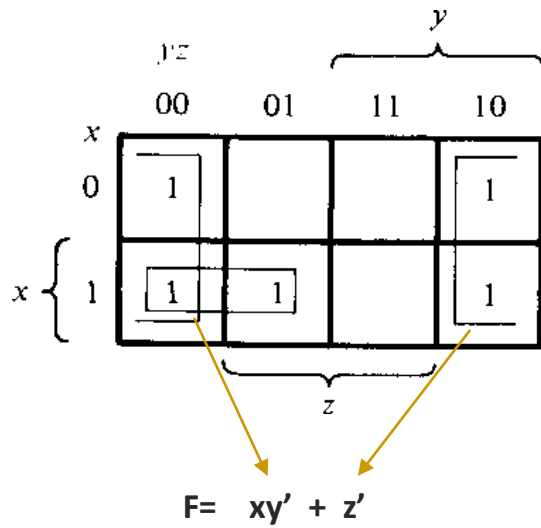


$$\begin{aligned}
 m_0 + m_2 + m_4 + m_6 &= x'y'z' + x'yz' + xy'z' + xyz' \\
 &= x'z'(y' + y) + xz'(y' + y) \\
 &= x'z' + xz' = z'(x' + x) = z'
 \end{aligned}$$

[Try it yourself]

- $F = \sum(0, 2, 4, 5, 6)$, Simplify it.

[Solution]



[Try it yourself:]

- $F = A'C + A'B + AB'C + BC$
 - a) Express it as SOP (i.e. Σ)
 - b) Simplify it using Kmap

[Solution]

■ (a) $F = \sum(1, 2, 3, 5, 7)$

(b) $F = C + A'B$

	BC	00	01	11	10
A	0		1	1	1
1			1	1	

To find $A'C$, coincide A' (first row) with C (middle 2 column)

To find $A'B$, coincide A' (first row) with B (last 2 column)

To find BC , coincide B (last 2 column) with C (middle 2 column)

To find $AB'C$, is m5(2nd row 2nd column)

Summary of grouping in 3-variable map

One square represents one minterm, giving a term of three literals.

Two adjacent squares represent a term of two literals.

Four adjacent squares represent a term of one literal.

Eight adjacent squares encompass the entire map and produce a function that is always equal to 1.

		yz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'

Four variable map

- Adjacent squares can be side by side or lie in 4 corner or top-bottom or left-right edge condition. Example, $m_0 - m_2$ are adjacent and similarly $m_3 - m_{11}$ are adjacent

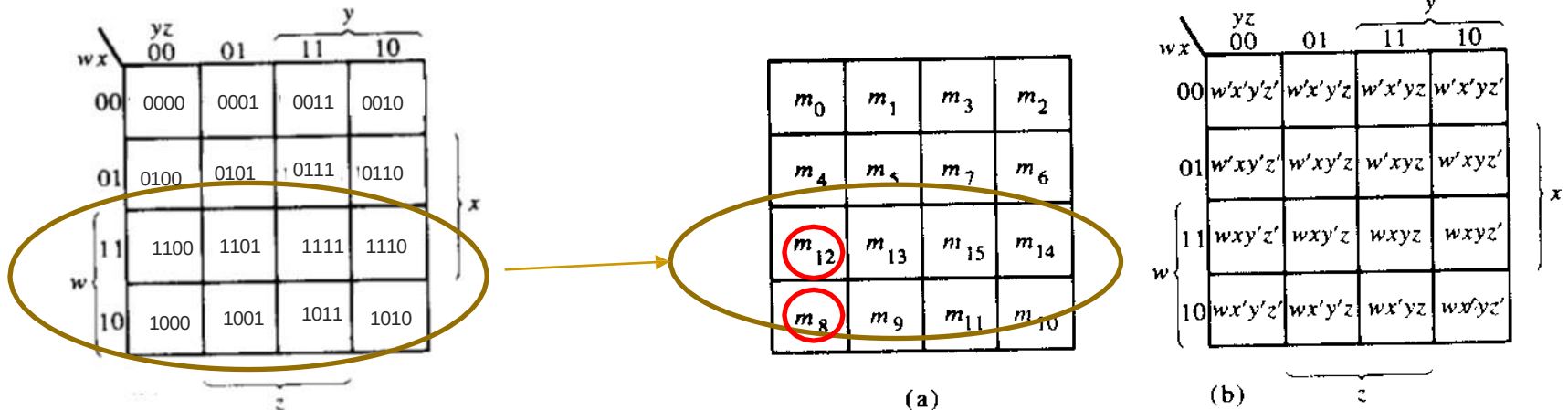
One square represents one minterm, giving a term of four literals.

Two adjacent squares represent a term of three literals.

Four adjacent squares represent a term of two literals.

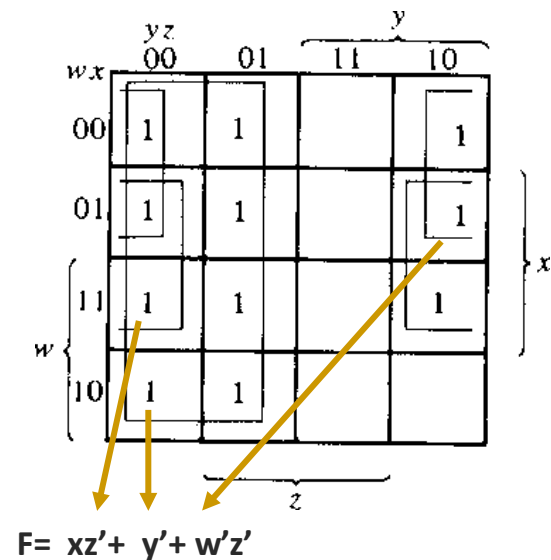
Eight adjacent squares represent a term of one literal.

Sixteen adjacent squares represent the function equal to 1.



- Simplify $F = \sum(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$
- Note: it is allowed to use the same '1' more than once.

wx \ yz	00	01	11	10
00	1	1		1
01	1	1		1
11	1	1		1
10	1	1		



[Try it yourself]

■ $F = A'B'C' + B'CD' + A'BCD' + AB'C'$

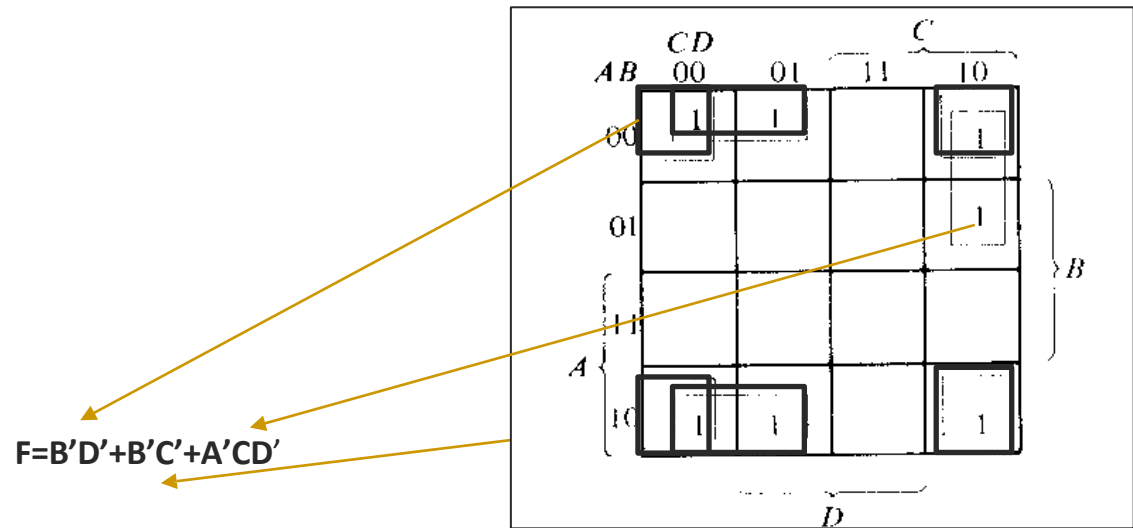
AB	CD		C	
	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

A { B }

D

[Solution]

■ $F = A'B'C' + B'CD' + A'BCD' + AB'C'$



Product of Sum (POS) simplification

- F is represented with the '1's
-> F' is represented with the '0's
-> (F')' = F so find F' and then find its complement using DeMorgan's law.
- In other word, '1's represent the minterms whereas '0's represent the maxterms.
- If Equation is provided in SOP format, mark the '1's and then mark the remaining with '0's. On the other hand, if Equation is provided in POS format, mark the '0's and then mark the remaining with '1's. Once marking is done, function can be simplified in SOP or POS format.

[SOP to POS]

- SOP $\rightarrow F = \sum(0, 1, 2, 5, 8, 9, 10, 11)$
- Combining '1's we get
 $F = B'D' + B'C' + A'C'D$
- Combining '0's we get
 $F' = AB + CD + BD'$
- Applying De-morgan
 $F = (A' + B')(C' + D')(B' + D) \leftarrow \text{POS}$

		<i>CD</i>		<i>C</i>	
		00	01	11	10
<i>AB</i>	00	1	1	0	1
	01	0	1	0	0
	11	0	0	0	0
	10	1	1	0	1

A { 00, 01, 11, 10 } *B*

{ 00, 01, 11, 10 } *D*

■ If SOP: $\sum(1,3,4,6)$

then POS: $\prod(0,2,5,7)$

So grouping '1's, $F = x'z + xz'$

Grouping '0's, $F' = xz + x'z'$
therefore, $F = (x' + z')(x + z)$

Truth Table of Function F

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

		yz		y	
		00	01	11	10
x	0	0	1	1	0
x	1	1	0	0	1
		z			

[Don't Care condition]

- Don't cares in a Karnaugh map, or truth table, may be either **1**s or **0**s, as long as we don't care what the output is for an input condition we never expect to see.
- We plot these cells with a special sign, 'X', among the normal **1**s and **0**s.
- When forming groups of cells, treat the don't care cell as either a **1** or a **0**, or ignore the don't cares. This is helpful if it allows us to form a larger group than would otherwise be possible without the don't cares.
- There is no requirement to group all or any of the don't cares. (i.e. do not make any group only consisting the don't cares). Only use them in a group if it simplifies the logic.

Simplify $F(w,x,y,z) = \sum(1,3,7,11,15)$ and don't care
 $d(w,x,y,z) = \sum(0,2,5)$

Solution: Even though 2 solutions are not same, but either
 1 is acceptable. Such scenario only applicable for Don't
 care scenarios!

		y			
		yz			
		00	01	11	10
w	wx				
	00	X	1	1	X
	01	0	X	1	0
	11	0	0	1	0
	10	0	0	1	0
		z			

x

(a) $F = yz + w'x'$

		y			
		yz			
		00	01	11	10
w	wx				
	00	X	1	1	X
	01	0	X	1	0
	11	0	0	1	0
	10	0	0	1	0
		z			

x

(b) $F = yz + w'z$

Complement of the function and POS considering don't cares

		y			
	yz	00	01	11	10
wx	00	X	1	1	X
	01	0	X	1	0
w	11	0	0	1	0
	10	0	0	1	0

Diagram illustrating a 4x4 Karnaugh map for the complement of the function F' . The map is labeled with variables w, x, y, z . The rows are labeled wx (00, 01, 11, 10) and the columns are labeled yz (00, 01, 11, 10). The map shows the values of F' for each combination of w, x, y, z . The values are: 0000: X, 0001: 1, 0010: 1, 0011: X, 0100: 0, 0101: X, 0110: 1, 0111: 0, 1100: 0, 1101: 0, 1110: 1, 1111: 0, 1000: 0, 1001: 0, 1010: 1, 1011: 0. The map is partitioned into groups: a group of four cells (0000, 0001, 0010, 0011) labeled x , a group of four cells (0100, 0101, 0110, 0111) labeled y , a group of four cells (1100, 1101, 1110, 1111) labeled z , and a group of four cells (1000, 1001, 1010, 1011) labeled w .

$$F' = z' + wy'$$

$$F = (z)(w' + y)$$

[

]

APPLICATIONS OF K-MAP

Example 1: Design and draw the circuit diagram of BCD to Excess-3 code converter

BCD Input				Excess-3 Output			
B3	B2	B1	B0	W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

Q1. Treat each W, X, Y and Z as a function and write them as SOP

W X
Y Z

Q2. Based on the SOP, draw the circuit diagram of BCD to Excess-3 code converter

Any value higher than 9 will result into don't care output

┌

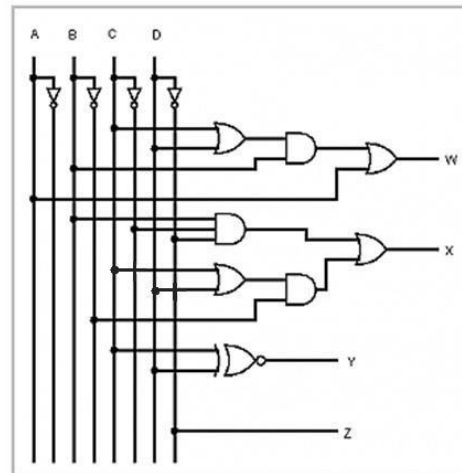
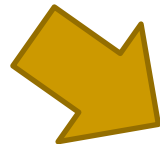
┐

For W

AB \ CD	00	01	11	10
00				
01		1	1	1
11	X	X	X	X
10	1	1	X	X

$$W = A + BC + BD$$

$$= A + B(C + D)$$



For X

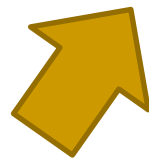
AB \ CD	00	01	11	10
00		1	1	1
01	1			
11	X	X	X	X
10		1	X	X

$$X = B\bar{C}\bar{D} + \bar{B}(C + D)$$

For Z

AB \ CD	00	01	11	10
00	1			1
01	1			1
11	X	X	X	X
10	1			X

$$Z = \bar{D}$$



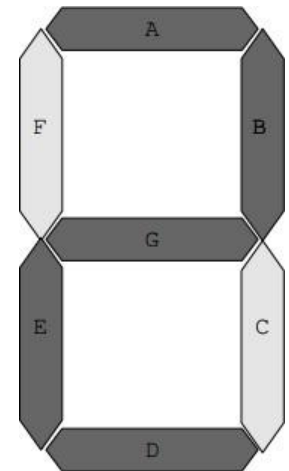
For Y

AB \ CD	00	01	11	10
00	1		1	
01	1		1	
11	X		X	X
10	1		X	X

$$Y = \bar{C}\bar{D} + CD$$

Example 2: 7 segment display board

Display digit	b8	b4	b2	b1	A	B	C	D	E	F	G
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	0	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1
off	1	0	1	0	0	0	0	0	0	0	0
off	1	0	1	1	0	0	0	0	0	0	0
off	1	1	0	0	0	0	0	0	0	0	0
off	1	1	0	1	0	0	0	0	0	0	0
off	1	1	1	0	0	0	0	0	0	0	0
off	1	1	1	1	0	0	0	0	0	0	0



┌

┐

Segment a

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	0	1	1	
!A	B	0	1	1	0	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$A + CD + B D + !B !D$$

Segment b

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	1	1	1	
!A	B	1	0	1	0	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$!B + !C !D + C D$$

Segment c

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	1	1	0	
!A	B	1	1	1	1	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$B + !C + D$$

Segment d

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	0	1	1	
!A	B	0	1	0	1	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$A + C !D + !B !D + !B C + B !C D$$

Segment e

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	0	0	1	
!A	B	0	0	0	1	
A	B	D	D	D	D	
A	!B	1	0	D	D	

$$C !D + !B !D$$

Segment f

			!C	!C	C	C
			!D	D	D	!D
!A	!B	1	0	0	0	
!A	B	1	1	0	1	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$A + B !C + !C !D + B !D$$

Segment g

			!C	!C	C	C
			!D	D	D	!D
!A	!B	0	0	1	1	
!A	B	1	1	0	1	
A	B	D	D	D	D	
A	!B	1	1	D	D	

$$A + B !C + B !D + !B C$$

[Try it by your self]

- A **car garage** has a front door and one window, each of which has a sensor to detect whether it is open. A third sensor detects whether it is dark outside. A security system for the garage follows this rule: the alarm rings if the **alarm switch is turned on** and **either the front door is not closed or it is dark and the side window is not closed**.



Solution

Switch On	Door	Window	Dark	AlarmRing
0	0	0	0	x
0	0	0	1	x
0	0	1	0	x
0	0	1	1	x
0	1	0	0	x
0	1	0	1	x
0	1	1	0	x
0	1	1	1	x
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

WDk

SDr

X	X	X	X
X	X	X	X
0	0	1	
0	0	1	0

$$\text{AlarmRing} = \text{WDk} + \text{WDr}$$

Assuming

AlarmRing = 1 if Alarm rings, 0 otherwise.

Switch On = 1 if alarm switch is on, 0 otherwise

Door = 1 if door is not closed, 0 otherwise.

Window = 1 if window is not closed.

Dark = 1 if it is dark outside, 0 otherwise.

K-Map Example

7

Gloria's Jeans wants to hire you to design their automated coffee machine. They want their machine to be able to make three of their most popular coffee flavors: 'Shot of espresso', "Latte" and "Mocha". Basic ingredient for any of these flavors are Brewed Coffee, full cream milk, sugar and coco powder.

Input and output of the automated coffee machine is as follows:

Input (4 variables)

C = 1 indicates presence of Brewed coffee.

M = 1 indicates presence of full cream milk

S = 1 indicates presence of sugar.

K = 1 indicates presence of Cocoa Powder

Output (3 variables)

E = 1 indicates **Shot of Espresso** is ready

L = 1 indicates **Latte** is ready

M = 1 indicates **Mocha** is ready.

Gloria's Jeans authorities has asked you to consider the following points when designing the system:

1. Without brewed coffee it is impossible to make any type of coffee.
 2. Brewed coffee and milk are essential for making Latte. Sugar is an optional (may or may not be there) ingredient. You do not need cocoa powder to make Latte.
 3. Brewed coffee, cocoa powder and milk are essential for making 'Mocha'. Sugar (may or may not be there) is an optional ingredient.
 4. 'Shot of Espresso' is just brewed coffee. Sugar is an optional ingredient. Cocoa powder and milk is not at all needed to make 'Shot of Espresso'.
- a) Using the above specification, prepare a truth table for the automated coffee machine.
- b) Using 4 variable Karnaugh-Map method, derive SOP expressions for all of the three flavors of coffee mentioned above.

K-Map Example



The ongoing EBOLA epidemic in the African continent has scared the authorities at Hajrat Shahjalal International Airport in Dhaka and they have asked the CSE department of BRACU to design a system that would help them to decide what to do with people coming in from African countries that are risk. The Airport authorities has approved the following specifications for the system:

Input (4 variables)

F = 0 indicates the passenger *DOES NOT* have FEVER.

H = 1 indicates the passenger *HAS* a SEVERE HEADACHE.

M = 0 indicates the passenger *DOES NOT* have MUSCLE PAIN.

A = 1 indicates the passenger *HAS ABDONIMAL (STOMACH) PAIN*.

Output (3 variables)

Q = 1 indicates *the passenger should be quarantined* (Kept locked up in a separate area and not let anyone come in contact with him or her so as to prevent EBOLA from spreading).

T = 1 indicates *the passenger is at RISK and needs to be tested*.

G = 1 indicates *the passenger is free to go*.

The Airport authorities has asked us to consider the following points when designing the system:

1. A passenger who has FEVER and MUSCLE PAIN and any one of the other two symptoms (STOMACH PAIN or SEVERE HEADACHE) should be QUARANTINED.
2. A passenger who has TWO OR MORE symptoms out of the four should be sent for testing.
3. A passenger who does not fall in any of the above criteria is free to go.

PLEASE NOTE THAT THIS POINTS ARE GIVEN IN ORDER OF PRIORITY ... so 1 HAS THE HIGHEST PRIORITY AND 3 HAS THE LOWEST.

- a) Using the above specification, prepare a truth table for the system.
(15 marks)
- b) Using 4 variable Karnaugh-Map method, derive SOP expressions for all of the three outputs.
(20 marks)

K-Map Example



The CSE Department at BRAC University is interested in designing a circuit that will allow them to automate the water dispensing machine in the department. The Chairperson has approved the following specification for the design of the system ...

Input (4 variables)

$H = 0$ indicates the temperature of the hot water compartment is *within the range*.

$C = 0$ indicates the temperature of the cold water compartment is *within the range*.

$L = 1$ indicates the *water level* inside the dispenser is *adequate*.

$F = 1$ indicates the *water filter* is in *good condition*.

Output (4 variables)

$W = 1$ indicates the heater is *on*.

$X = 1$ indicates the cooler is *on*.

$Z = 1$ indicates *water in the dispenser is drinkable*.

$T = 1$ indicates *the supply tap is open*.

The directors have asked us to consider the following points when designing the system:

1. If the temperature of the hot water compartment is outside the range, the heater should be TURNED OFF.
2. If the temperature of the cold water compartment is outside the range, the cooler should be TURNED OFF.
3. If the water level is ADEQUATE, the supply tap should be TURNED OFF.
4. If the water filter is in GOOD CONDITION, then the water in the dispenser is DRINKABLE.

a) Using the above specification, prepare a truth table for the system.

b) Using 4 variable Karnaugh-Map method, derive SOP expressions for all of the three outputs.

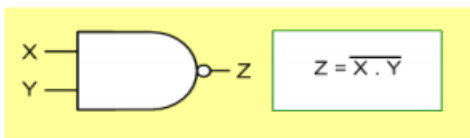
[

]

UNIVERSAL LOGIC GATES

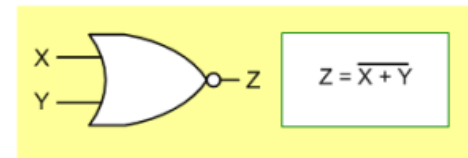
NAND

X	Y	NAND
0	0	1
0	1	1
1	0	1
1	1	0



NOR

X	Y	NOR
0	0	1
0	1	0
1	0	0
1	1	0

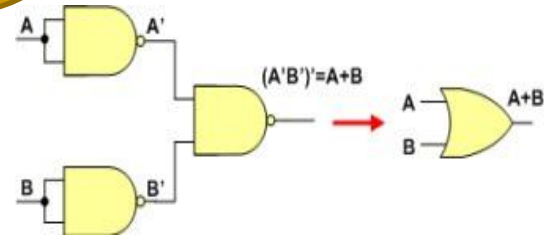


[Using NAND]

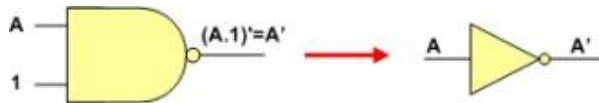
NOT Gate



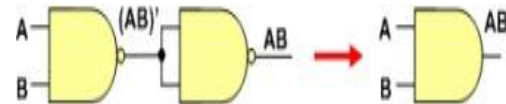
OR Gate



NOT Gate



AND Gate

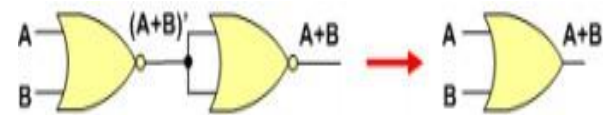


[Using NOR]

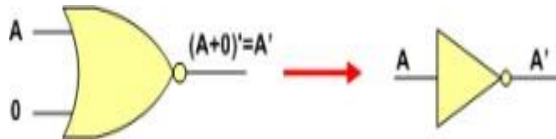
NOT Gate



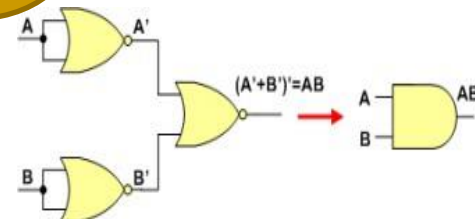
OR Gate



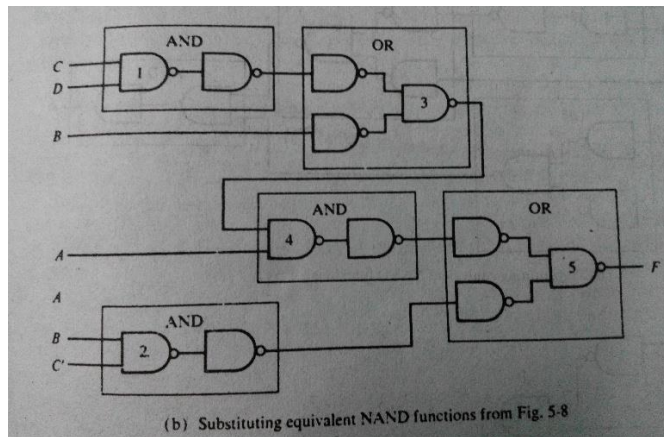
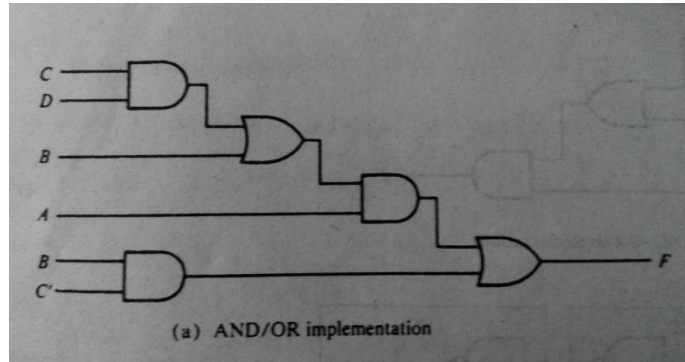
NOT Gate



AND Gate

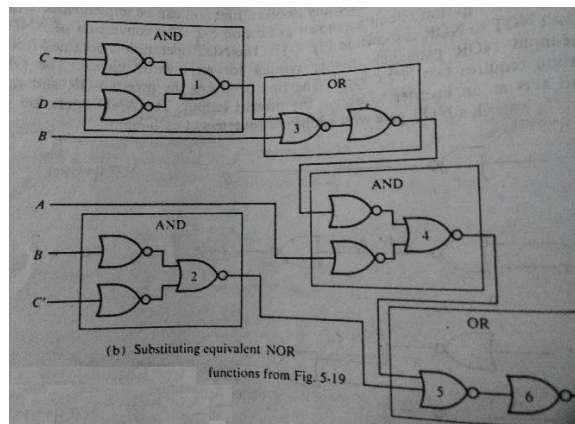
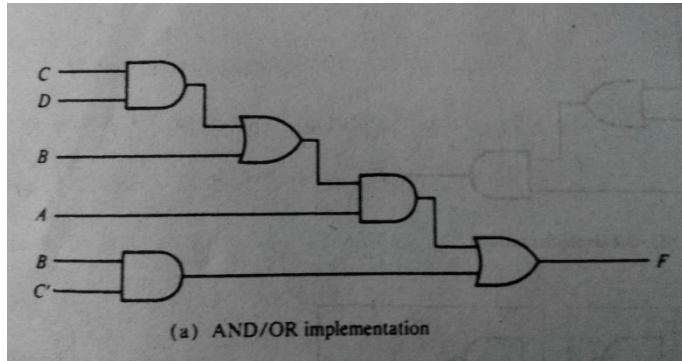


Use NAND
to
represent
 $F = A(B + CD) + BC'$
+BC'





Use NOR
to
represent
 $F=A(B+CD)+BC'$



[A Must Self study from the book]

- Morris Mano: Chapter 3.6 (NAND and NOR implementation)to Chapter 3.7 (Other two level implementation)